

TP 12

Dictionnaires et Piles – Sujet

Autour des listes et des dictionnaires

Soit une liste `L` de chaînes de caractères.

Question 1 Implémenter une fonction `comptage_lettres(L:[str]): -> {str:int}` renvoyant le dictionnaire de comptage dont les clefs sont les lettres de la liste `L` et les valeurs le nombre d'occurrences.

Exemple –

Si `L=['a', 'b', 'a', 'a', 'c', 'e', 'z', 'z', 'a', 'a']`, `comptage_lettres(L)` devra renvoyer `{'a':5, 'b':1, 'c':1, 'e':1, 'z':2}`.

Soit une liste `L` d'entiers.

Question 2 Implémenter une fonction `comptage(L:[int]): -> {int:int}` renvoyant le dictionnaire de comptage dont les clefs sont les entiers de la liste `L` et les valeurs le nombre d'occurrences.

Exemple –

Si `L=[0, 4, 6, 1, 4, 3, 3, 5, 3, 8]`, `comptage(L)` devra renvoyer `{0:1, 4:2, 6:1, 1:1, 3:3, 5:1, 8:1}`.

Question 3 Implémenter une fonction `suppression_doublons(L:[int]): -> []` renvoyant une liste où tous les doublons auront été supprimés et ce, en conservant l'ordre initial¹.

Exemple –

Si `L=[0, 4, 6, 1, 4, 3, 3, 5, 3, 8]`, `suppression_doublons(L)` devra renvoyer `L=[0, 4, 6, 1, 3, 5, 8]`.

1: On pourra remarquer que la liste demandée correspond à la liste des clés du dictionnaire de comptage.

Autour des piles

Dans le cadre cet exercice, les piles dont des `deque` importé du module `collections`². On ne s'autorise que les opérations suivantes :

- `pile = deque()` ou `pile = deque([3,2,1,2,3])` pour créer une pile;
- `len(pile) == 0` pour vérifier si une pile est vide;
- `pile.append('Truc')` pour ajouter un élément dans la pile;
- `pile.pop()` pour supprimer et renvoyer le sommet de la pile.

Il n'est donc pas possible d'utiliser l'instruction `for i in range(len(pile))`.

Question 4 Implémenter la fonction `somme(pile)` renvoyant la somme des éléments d'une pile d'entiers. La pile initiale devra rester inchangée.

Question 5 Implémenter la fonction `maximum(pile)` renvoyant le maximum des éléments d'une pile d'entiers. La pile initiale devra rester inchangée.

2: `from collections import deque`

Question 6 Implémenter la fonction `pop_maximum(pile)` renvoyant le maximum des éléments d'une pile d'entiers. La pile initiale sera donc la même à l'exception du maximum le plus haut. Si le maximum apparaît plusieurs fois, on les supprimera tous.

Gérer les stocks de composants pour réaliser des drones

L'entreprise **SuperDrone** réalise le montage et la vente de drones.

Les composants utiles pour réaliser un drone sont :

- ▶ le châssis;
- ▶ les 4 moteurs;
- ▶ les 4 hélices;
- ▶ la batterie;
- ▶ le contrôleur de vol;
- ▶ ESC 4 en 1 pour les 4 moteurs (Electronic Speed Controller);
- ▶ la plaque de distribution de puissance (PDB Power Distribution Board).

Des composants en option sont aussi disponibles (caméra, radiocommande, chargeur, buzzer, leds...) et ne seront pas traités ici.



Dans le fichier `drone.py` donné, vous trouverez 4 dictionnaires dont les clés sont les noms des composants et les valeurs représentent le nombre de composants nécessaire :

```
1 drone={'moteur':4, 'chassis':1, 'contrôleurVol':1, 'ESC4en1':1, 'batterie':1, '
    helice':4, 'plaqueDeDistribution':1}
2 stock={'chassis':0, 'moteur':25, 'helice':36, 'contrôleurVol':12, 'ESC4en1':8, '
    batterie':20, 'plaqueDeDistribution':7}
3 limiteMin={'chassis':2, 'moteur':8, 'helice':8, 'contrôleurVol':2, 'ESC4en1':2, '
    batterie':2, 'plaqueDeDistribution':2}
4 limiteMax={'chassis':15, 'moteur':60, 'helice':60, 'contrôleurVol':15, 'ESC4en1':
    15, 'batterie':30, 'plaqueDeDistribution':15}
```

- ▶ `drone`, correspond aux composants nécessaires à la réalisation d'un drone. Les clés étant les composants et les valeurs le nombre de composant pour un drone;
- ▶ `stock`, correspond au stock à l'instant considéré;
- ▶ `limiteMin`, correspond aux valeurs limites basses du stock pour déclencher une commande;
- ▶ `limiteMax`, correspond aux valeurs limites hautes pour reconstituer le stock et pour définir le nombre de composants à commander.

L'objectif est l'écriture d'un programme qui permette de générer les commandes de composants utiles pour assurer la réalisation des drones attendus par les clients sans avoir de rupture de stock. On utilisera des objets de type `dict`.

Réaliser un drone

Question 7 Écrire la fonction `realiser1Drone(drone:dict, stock:dict)` qui prend pour argument les dictionnaires `drone` et `stock` et qui renvoie un booléen, `True` si le stock est suffisant pour réaliser un drone, `False` sinon.

Gérer le stock de composants

Question 8 Écrire la fonction `destocker(D:dict, S:dict)` qui prend pour argument les dictionnaires `D` des composants du drone et `S` du stock et qui retire du stock le nombre de composants utiles pour réaliser un drone. Cette fonction ne renvoie rien. Le dictionnaire `S` est modifié par effet de bord.

Lorsque le stock devient insuffisant, une commande est passée et le stock est ré-évalué.

Question 9 Écrire la fonction `stocker(C:dict, S:dict)` qui prend pour argument les dictionnaires `C` correspondant à la commande et `S` du stock et qui ajoute au stock le nombre de composants commandés. Cette fonction ne renvoie rien. Le dictionnaire `S` est modifié par effet de bord, le dictionnaire `C` n'est pas modifié.

Passer une commande

Une commande est passée quand le stock d'un composant est à la limite basse (valeur incluse). Les composants qui n'ont pas atteint la limite basse ne sont pas commandés.

Question 10 Écrire la fonction `commanderComposant(S:dict, limiteMin:dict, limiteMax:dict)` qui prend pour argument les dictionnaires `S` du stock, `limiteMin` et `limiteMax` et qui renvoie un dictionnaire `commande` permettant de reconstituer le stock.

Gestion automatique

Dans la semaine, l'entreprise `SuperDrone` reçoit les commandes de drones de 4 clients sous la forme d'une liste, `listeCommande` `[3, 1, 5, 2]`.

Question 11 Écrire la fonction `satisfaireClient(listeCommande:list, drone:dict, stock:dict, limiteMin:dict, limiteMax:dict)` qui prend pour argument une liste de commande de drones, les dictionnaires `drone`, `stock`, `limiteMin` et `limiteMax` et qui affiche l'état du stock après chaque réalisation d'un drone ainsi que les commandes successives.

Snakes and ladders : le jeu

Présentation du jeu

Le jeu *serpents et échelles* est un jeu de société où on espère monter les échelles en évitant de trébucher sur les serpents. Il provient d'Inde et est utilisé pour illustrer l'influence des vices et des vertus sur une vie.

Le plateau

- Le plateau comporte 100 cases numérotées de 1 à 100 en boustrophédon³ : le 1 est en bas à gauche et le 100 est en haut à gauche ;
- des serpents et échelles sont présents sur le plateau : les serpents font descendre un joueur de sa tête à sa queue, les échelles font monter un joueur du bas de l'échelle vers le haut.

Extrait du travail de T. Kovaltchouk - UPSTI

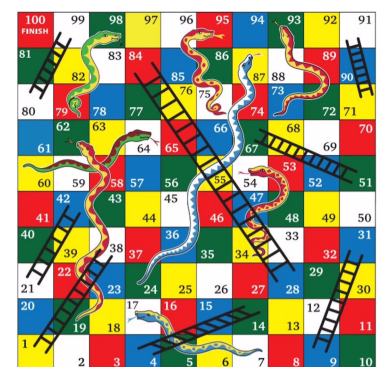


FIGURE 1 – Exemple d'un plateau de serpents et échelles

3: à la manière du bœuf traçant des sillons, avec alternance gauche-droite et droite-gauche

Déroulement

- ▶ Chaque joueur a un pion sur le plateau. Plusieurs pions peuvent être sur une même case. Les joueurs lancent un dé à tour de rôle et ils avancent du nombre de cases marqués sur le dé. S'ils atterrissent sur un bas d'échelle ou une tête de serpent, ils vont directement à l'autre bout;
- ▶ les joueurs commencent sur une case 0 hors du plateau : la première case où mettre leur pion correspond donc au premier lancer de dé;
- ▶ le premier joueur à arriver sur la case 100 a gagné;
- ▶ il existe 3 variantes quand la somme de la case actuelle et du dé dépasse 100 :
 - le rebond : on recule d'autant de cases qu'on dépasse;
 - l'immobilisme : on n'avance pas du tout si on dépasse;
 - la fin rapide : on va à la case 100 quoi qu'il arrive.

On utilisera les notations suivantes pour les complexités : N_{cases} , le nombre de cases du plateau (100), et N_{SeE} la somme du nombre de serpents et du nombre d'échelle (16 dans notre exemple).

Simulation du jeu

`randint(a, b)` method of `random.Random` instance Return random integer in range `[a, b]`, including both end points.

`choice(seq)` method of `random.Random` instance Choose a random element from a non-empty sequence.

Question 12 Écrire une fonction `lancerDe()` -> `int` qui renvoie un nombre entier compris entre 1 et 6 en utilisant une fonction du module `random`.

Les serpents et les échelles sont représentés par un dictionnaire `dSeE` tel que, pour une case de départ numérotée `i`, `dSeE[i]` donne le numéro de la case d'arrivée.

Avec l'exemple de la figure 1, on a :

```
1 dSeE = { 1: 38, 4: 14, 9: 31, 17: 7, 21: 42, 28: 84, 51: 67, 54: 34,
2         62: 19, 64: 60, 71: 91, 80: 99, 87: 24, 93: 73, 95: 75, 98: 79}
```

Question 13 Écrire la fonction `caseFuture(case: int) -> int` qui prend en argument le numéro de la case et qui renvoie le numéro de la case où va se trouver le joueur en atterrissant sur la case numérotée `case`. Par exemple, `caseFuture(5)` renvoie 5 (c'est un numéro de case stable), `caseFuture(1)` renvoie 38 (c'est un numéro de case avec échelle) et `caseFuture(17)` renvoie 7 (c'est un numéro de case avec une tête de serpent).

Question 14 Quelle est la complexité de cette fonction ?

Question 15 Écrire une fonction `avanceCase(case: int, de: int, choix: str) -> int` qui renvoie la case d'arrivée lorsqu'on part de la case `case` et qu'on a comme résultat au lancer du dé la valeur `de`. La variable `choix` est une chaîne de caractère correspondant à la stratégie de fin différente : "r" pour le rebond, "i" pour l'immobilisme et "q" pour une fin rapide. .

Question 16 Écrire une fonction `partie(choix: str) -> [int]` qui lance une partie à un joueur et renvoie la liste successive des cases visitées sur le plateau. Elle commencera donc forcément par 0 et finira forcément par 100. Le choix du mode de fin est en argument, de façon similaire à la question précédente.

Plus court chemin

On souhaite, dans cette partie, utiliser un algorithme glouton pour trouver la partie la plus courte.

Question 17 Écrire une fonction `casesAccessibles(case: int) -> [int]` qui renvoie la liste des 6 cases accessibles pour la case donnée en entrée. Vous utiliserez la fonction `avanceCase` de la question 15. La liste renvoyée `cases` doit avoir le codage suivant : `cases[i]` doit correspondre à la case d'arrivée avec le résultat de dé `i+1` (donc la liste retournée doit toujours avoir une longueur de 6). On prendra l'option de fin rapide.

Question 18 Écrire une fonction `meilleurChoix(case: int) -> int` qui renvoie la meilleure case accessible depuis `case`. Il est interdit d'utiliser la fonction `max` dans cette question.

L'algorithme glouton consistera à choisir la valeur du dé permettant de maximiser son déplacement à chaque coup.

Question 19 Écrire une fonction `partieGloutonne() -> [int]` qui renvoie la liste des cases par lesquelles passe le pion dans l'algorithme glouton.

Cette dernière fonction nous renvoie `[0, 38, 44, 50, 67, 91, 97, 100]`.